LAB: Color Image Segmentation - Magic Cloak

Date: 2025-APRIL-15

Author: HWANG MooYeol 22000814

Github: [-]

Demo Video: YouTube

Introduction

1. Objective

- **Goal**: Invisible Cloak

  The goal of this project is to create a basic OpenCV program that makes a person wearing a cloak appear invisible.

  This is done by detecting the cloak based on its color and replacing it with the background image, so it looks like the person disappears.

  The program uses simple image processing techniques like color detection, masking, and background subtraction to achieve this visual effect. This project helps demonstrate how computer vision can be used to create optical illusions and can be a starting point for more advanced augmented reality systems.

2. Preparation

Software Installation

- OpenCV 4.9.0, Visual Studio 2022

Algorithm

1. Overview

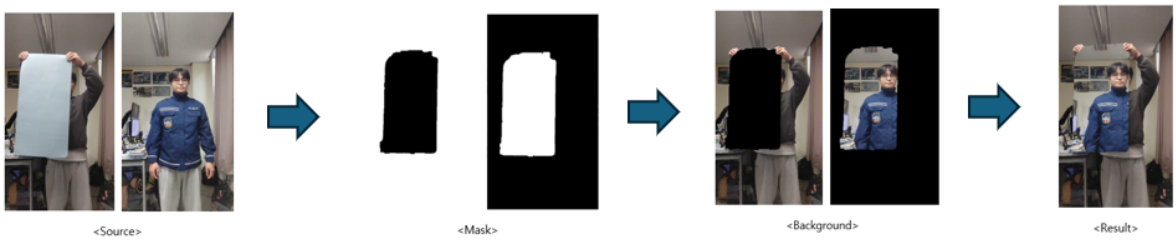

Figure 1. Overview

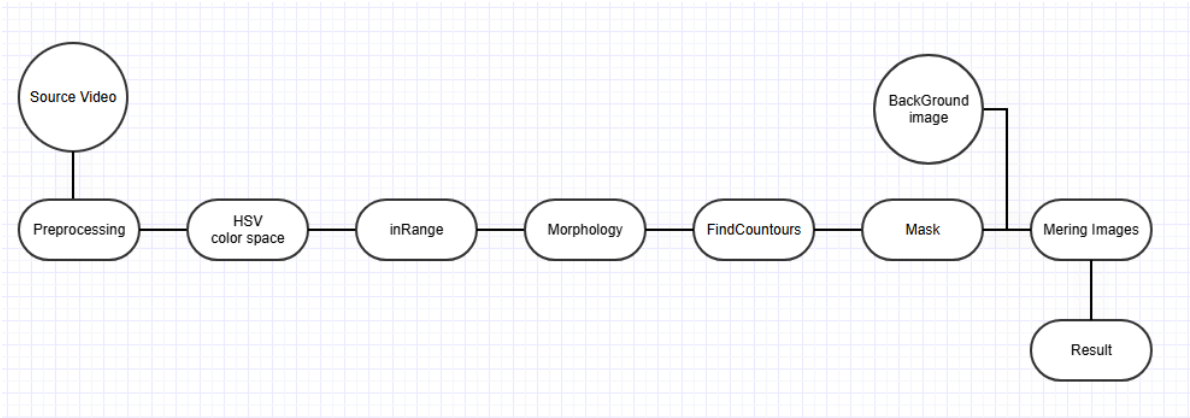

Figure 2. Flowchart

2. Procedure

- Design algorithms for the given mission
  - Include a flowchart to explain the algorithm flow.
- You should apply image processing algorithms, which you have learnt in class, as much as possible.
- If you want to use other algorithms that were not covered in class, then you should briefly explain how that algorithm works.
- Work on Sample #1. You should explain and show each necessary step including
  - Pre-Processing (filtering etc)
  - Color Segmentation
  - Masking
  - Merging Images and more
- Work on Sample #2.
- Here, you can only show the final output image results in the report.
- Also, create a short demo video (about 10sec) and upload online.
- You need to make the appropriate title page in the demo video
  - Showing Class name, date, student name, title etc

Convert RGB to HSV space

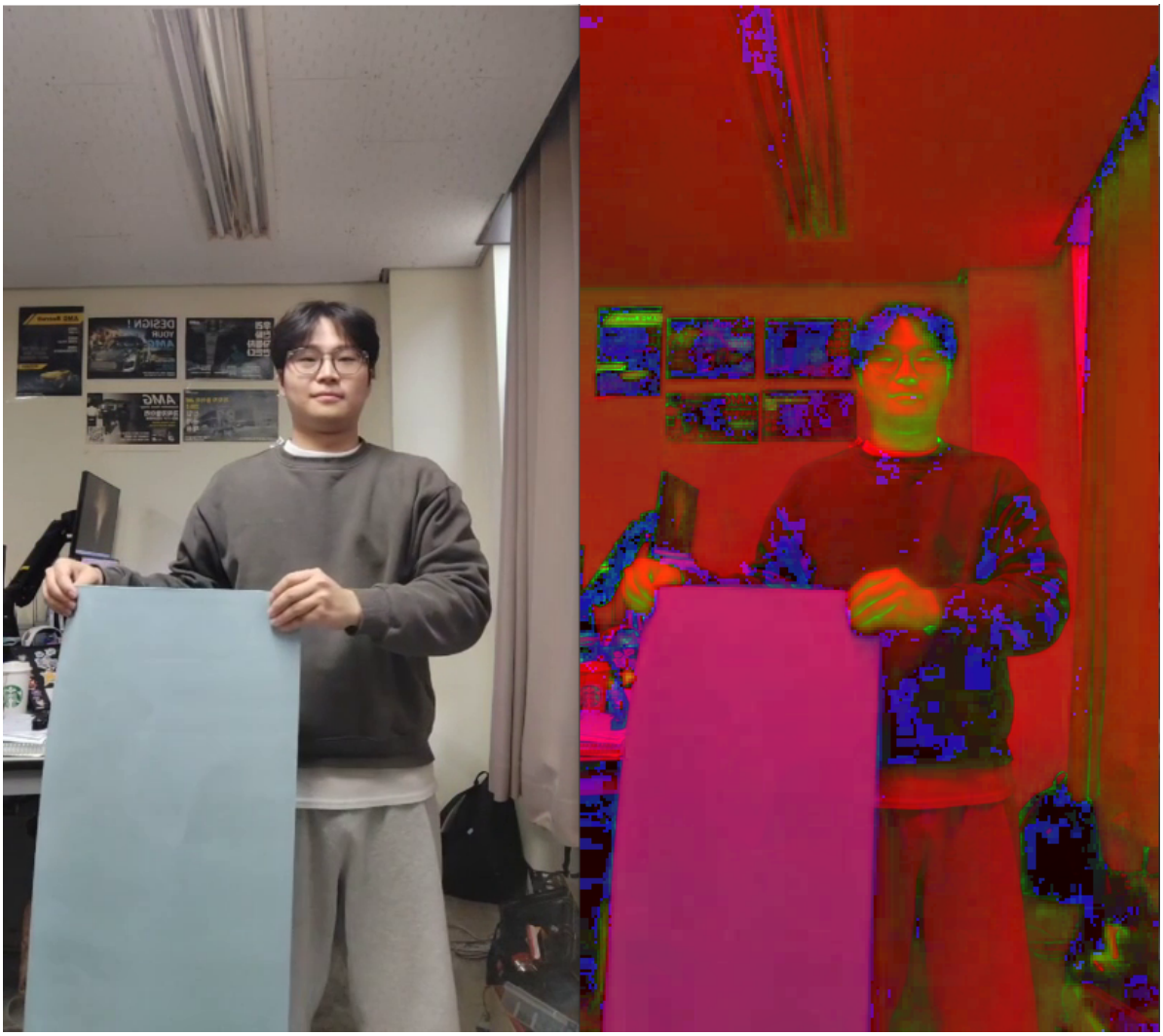

Figure 3. HSV color space

Used the `cvtColor` function `COLOR_BGR2HSV` to convert HSV .

```
cvtColor(src, hsv, COLOR_BGR2HSV);
```

Color Segmentation - InRange

Figure 4. HSV values for InRange

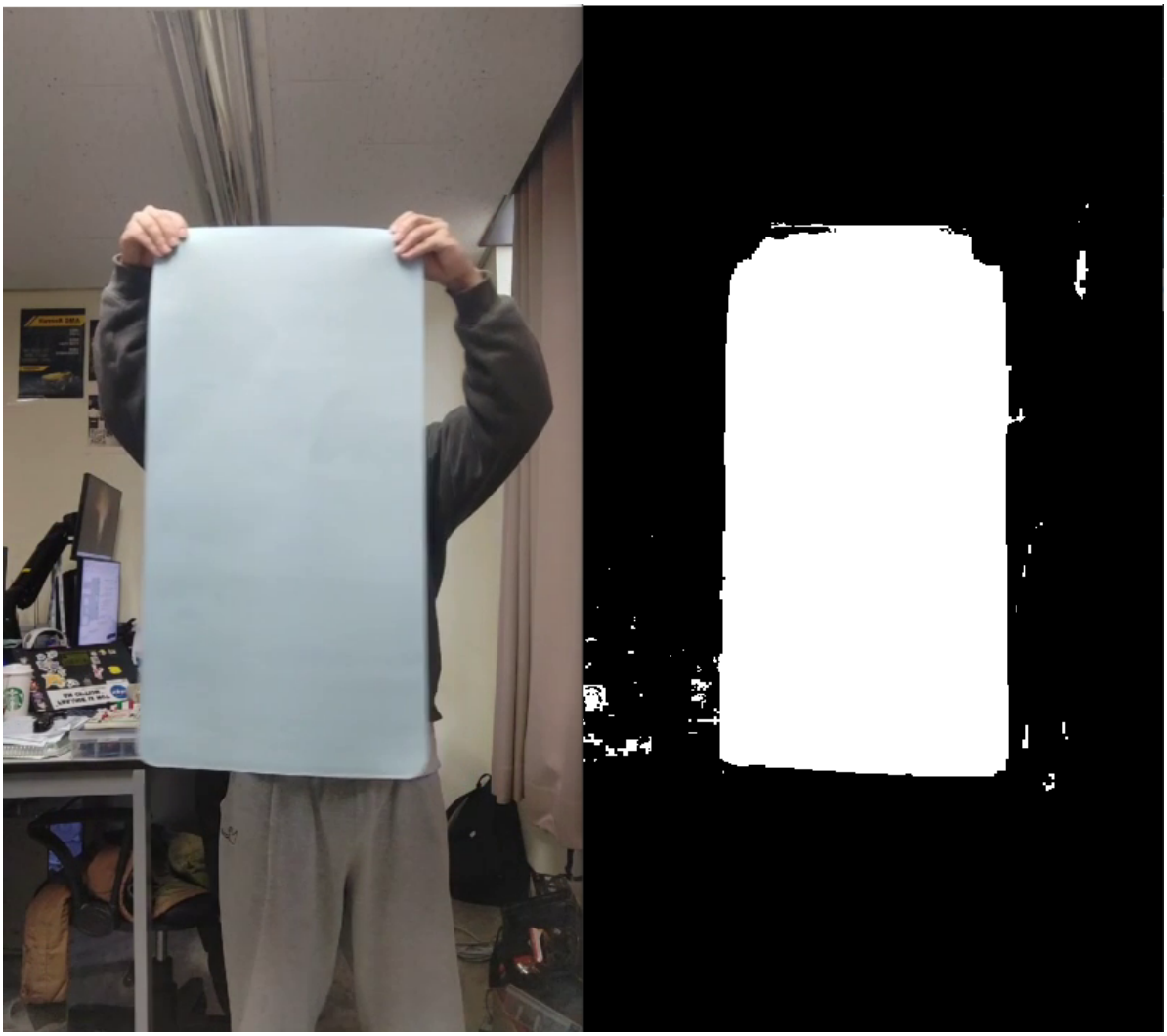

Figure 5. InRange result

Used the `inRange` function to segment color .

```
void cv::inRange(
    InputArray src,
    InputArray lowerb,
    InputArray upperb,
    OutputArray dst
);

int hmin = 28, hmax = 110, smin = 4, smax = 161, vmin = 115, vmax = 250;
inRange(hsv, Scalar(hmin, smin, vmin), Scalar(hmax, smax, vmax), dst);
```

Thresholding and Morphology

Figure 6. Result after Morphology(Opening)

Opening algorithm to eliminate noise.

Used the `morphologyEx` function to eliminate noise.

```
void cv::morphologyEx(
    InputArray src,
    OutputArray dst,
    int op,
    InputArray kernel,
    Point anchor = Point(-1, -1),
    int iterations = 1,
    int borderType = BORDER_CONSTANT,
    const Scalar& borderValue = morphologyDefaultBorderValue()
);

morphologyEx(dst, dst, MORPH_OPEN, getStructuringElement(MORPH_RECT, Size(4,
4)));
```

Finding Contours

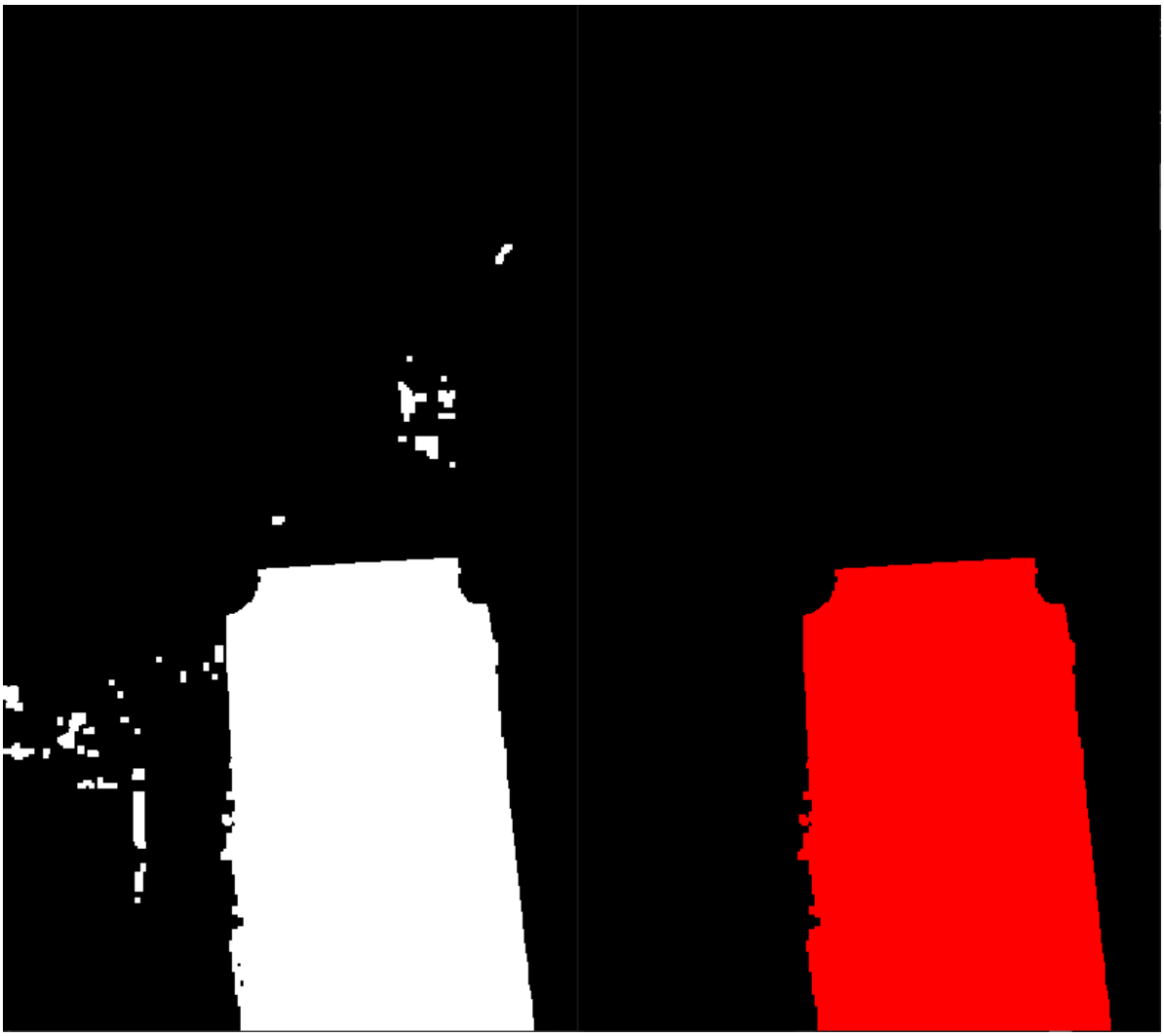

Figure 7. Finding Largest area with contour

Area was compared for all contours found to find the largest area.

```
vector<vector<Point>> contours;
findContours(dst, contours, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);
if (!contours.empty())
{
    double maxArea = 0;
    int maxIdx = 0;
    for (int i = 0; i < contours.size(); i++) {
        double area = contourArea(contours[i]);
        if (area > maxArea) {
            maxArea = area;
            maxIdx = i;
        }


    }
    mask = Mat::zeros(video.size(), CV_8UC3);
    drawContours(mask, contours, maxIdx, Scalar(255, 255, 255), FILLED);
    dilate(mask, mask, Mat(), Point(-1, -1), 5);
}
```

Making Masking

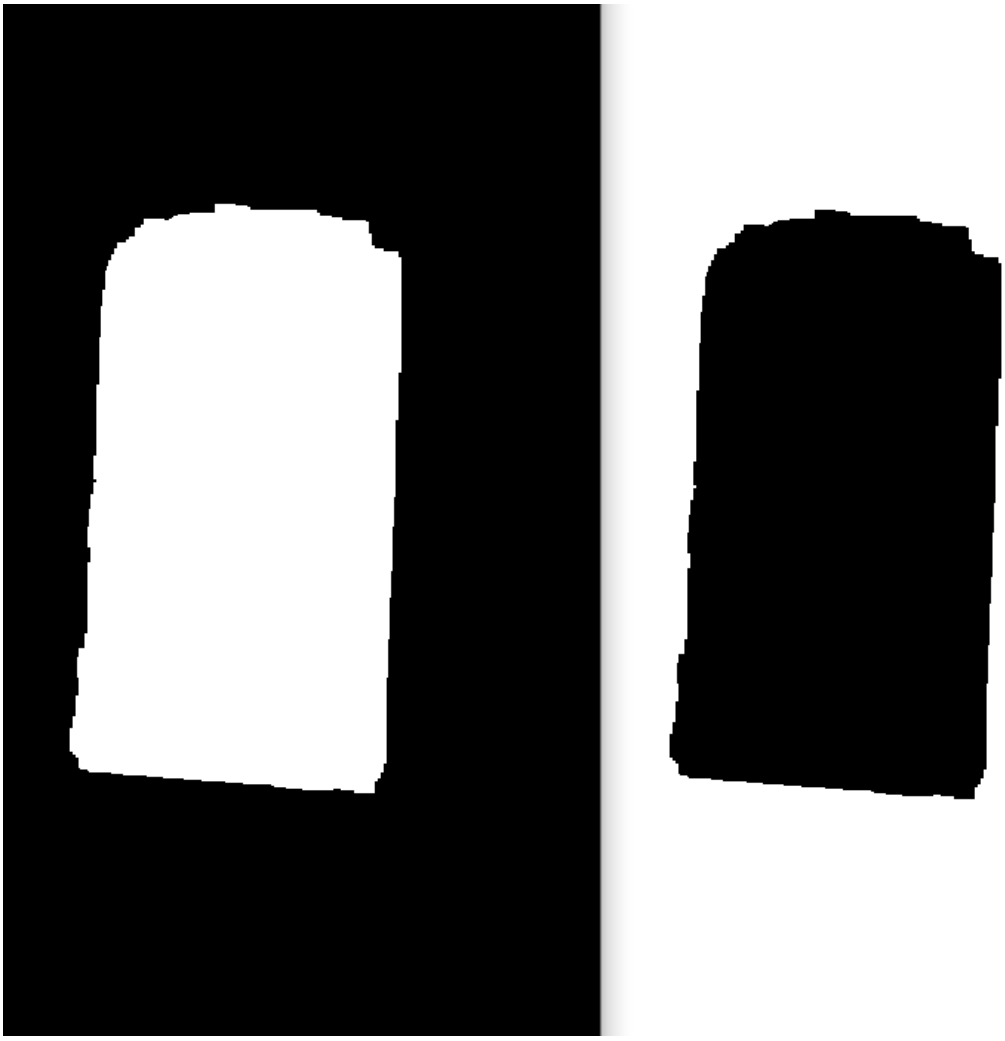

Figure 8. Mask and Inverse Mask

Based on the largest area of the contour, a mask is made and an inverse mask is also made for the background

The `bitwise_not` function was used to inverse mask image.

```
void cv::bitwise_not(
    InputArray src,
    OutputArray dst,
);

mask = Mat::zeros(video.size(), CV_8UC3);
drawContours(mask, contours, maxIdx, Scalar(255, 255, 255), FILLED);
dilate(mask, mask, Mat(), Point(-1, -1), 5);
imshow("mask", mask);
bitwise_not(mask, mask_inv);
imshow("maskinv", mask_inv);
```

Extracting and Merging Images

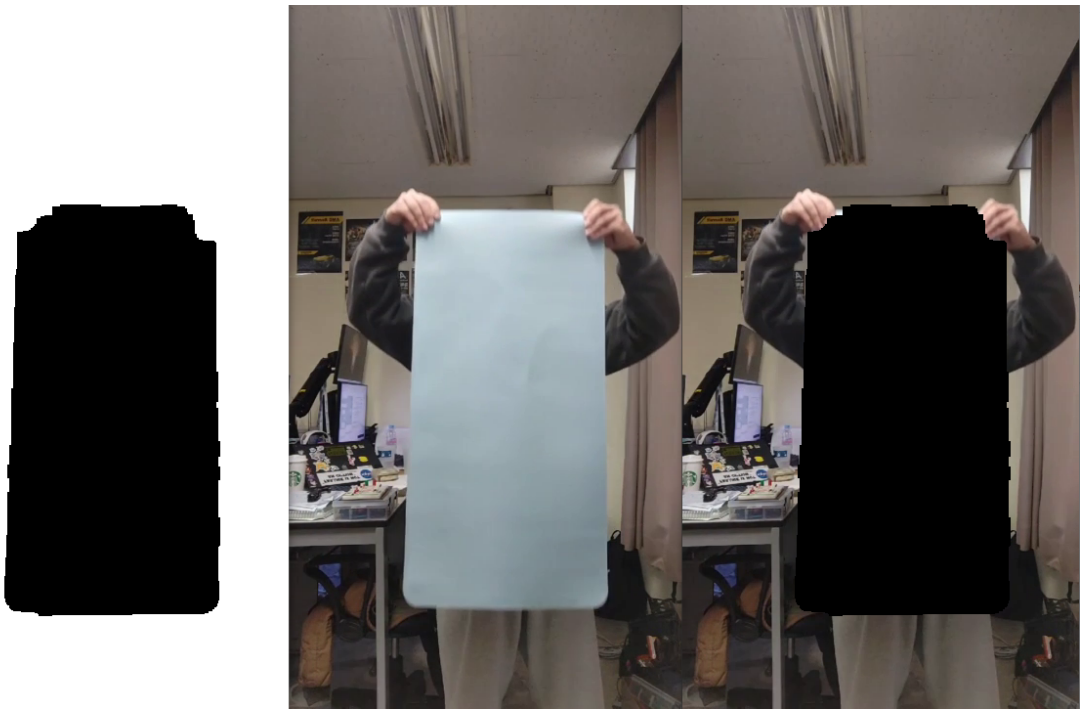

Figure 9. Background extract

Figure 10. ColorArea extract

Figure 11. Result BY Merging

The images were merged using the bitwise operator `&` and `+`.

```
background = bg & mask;

current = video & mask_inv;

result = current + background;
```

Result and Discussion

1. Final Result

By segmenting the color of the patch, it was possible to make a final completion that merges the two images into one. The part covered with the patch was intended to change the wearing clothes, it can be seen that it was well implemented.

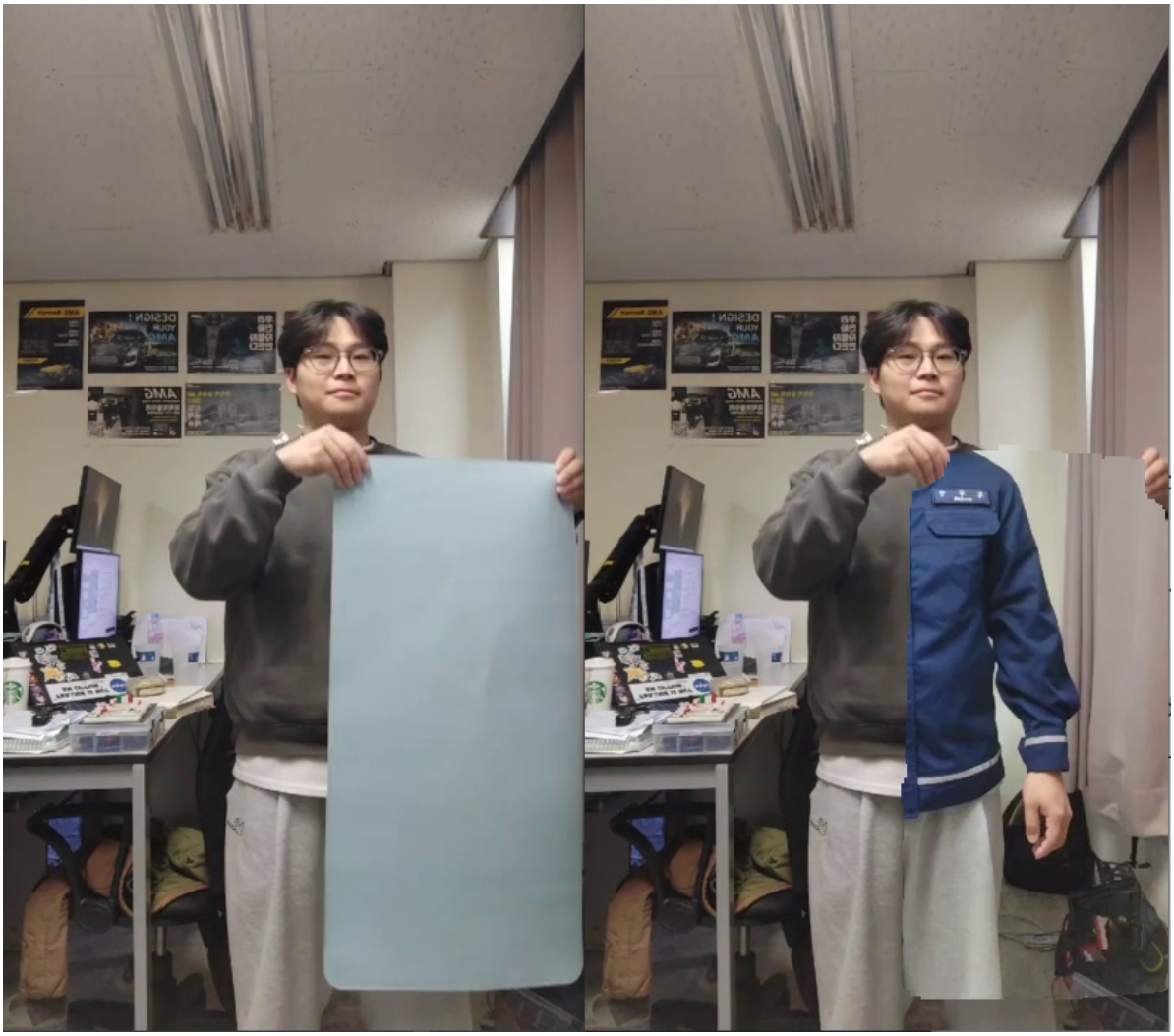

Figure 12. Final Result for Magic cloak

Demo Video

[YouTube]

2. Discussion

By applying a method of detecting colors by specifying the value range in the HSV color space. Realized that the same object can appear with different HSV values depending on the lighting conditions, light environment was found to be very important.

Compared to the RGB color space, the HSV color space is easier to handle by separating the color, brightness, and saturation of colors, so it was found that HSV can be more advantageous in image processing, where colors can be easily changed according to brightness changes.

During the image preprocessing stage, I observed that using a sharpening filter can make edges clearer, which in turn helps distinguish hue values more precisely in the HSV space.

How to deal with noise is also considered a very important factor when detecting a color area.

Conclusion

In this project, we implemented an "Invisible Cloak" effect using basic OpenCV techniques, where a cloak of a specific color is detected and replaced with the background to create the illusion of invisibility. This effect was achieved through fundamental image processing steps such as color detection, masking, and background subtraction.

By applying color detection using specified value ranges in the HSV color space, it was observed that the same object can appear with different HSV values depending on lighting conditions. This highlights the critical importance of the lighting environment in color-based image processing. Compared to the RGB color space, HSV provides a more intuitive way to isolate and manipulate colors by separating hue, saturation, and brightness. This makes HSV particularly advantageous in scenarios where brightness variation significantly affects color representation.

Furthermore, during the image preprocessing stage, the use of sharpening filters was found to enhance edge clarity, which in turn helps in more precise hue differentiation in the HSV space. Lastly, handling noise effectively is also a key consideration in accurately detecting color regions and maintaining the quality of the visual effect.

Overall, this project highlights the potential of basic OpenCV techniques in creating visual effects, serving as a stepping toward more advanced augmented reality applications.

Appendix

```
#include <opencv2/opencv.hpp>

using namespace cv;
using namespace std;

int frame_cnt;
int frame = 0;
int hmin = 28, hmax = 110, smin = 4, smax = 161, vmin = 115, vmax = 250;
Mat frame1, frame2, video, hsv, dst, mask, mask_inv, background, current,
result, bg;

int main(int ac, char** av) {

    VideoCapture cap("LAB2.mp4");
    VideoCapture bg1("background.mp4");

    frame_cnt = cap.get(CAP_PROP_FRAME_COUNT);

    if (!cap.isOpened())
    {
        printf("Can't open the camera");
        return -1;
    }

    bg1 >> frame1;      // 1st frame
    imshow("img1", frame1);
    bg = frame1;
```

```
    while (frame < frame_cnt - 1)
    {
        cap >> video;
        frame += 1;

        cvtColor(video, hsv, COLOR_BGR2HSV);
        inRange(hsv, Scalar(hmin, smin, vmin), Scalar(hmax, smax, vmax), dst);
        //imshow("InRange", dst);
        morphologyEx(dst, dst, MORPH_OPEN, getStructuringElement(MORPH_RECT,
Size(4, 4)));
        //imshow("mor", dst);

        vector<vector<Point>> contours;

        findContours(dst, contours, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);
        if (!contours.empty())
        {
            double maxArea = 0;
            int maxIdx = 0;
            for (int i = 0; i < contours.size(); i++) {
                double area = contourArea(contours[i]);
                if (area > maxArea) {
                    maxArea = area;
                    maxIdx = i;
                }
            }
            mask = Mat::zeros(video.size(), CV_8UC3);
            drawContours(mask, contours, maxIdx, Scalar(255, 255, 255), FILLED);
            dilate(mask, mask, Mat(), Point(-1, -1), 5);
            //imshow("mask", mask);
            bitwise_not(mask, mask_inv);
            //imshow("maskinv", mask_inv);

            background = bg & mask;
            //imshow("bg", background);
            current = video & mask_inv;
            //imshow("current", current);
            result = current + background;
            imshow("result", result);
        }


        if (video.empty())
        {
            printf("empty image");
            return 0;
        }

        imshow("camera img", video);

        if (waitKey(25) == 27)
            break;
    }


    return 0;
}
```